

MINI PROJECT REPORT

Motion Detection Using CCTV Camera with Python

Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

Bachelor of Commerce (B.Com) – Computers

Submitted By

Student Name: _Gudala.Ruchitha

Under the Guidance of : Mrs Sai Nikhitha

Department of Commerce (Computer Applications)

College Name: vagdevi degree college nizamabad

Affiliated to: Telangana_University

Academic Year: 2024-2027

Internship Mini Project Repo

Acknowledgement

I express my sincere gratitude to our Mentor, and all the faculty members of the Department of Brainic Academy (Computer Applications) for providing me with the opportunity to complete this mini project successfully.

I would like to express my heartfelt thanks to my project guide, for providing valuable guidance, continuous encouragement, and constructive suggestions throughout the development of this project. Their support and motivation helped me complete this project successfully.

I am also thankful to my internship organization (if applicable) for providing me with the opportunity to work on the project "**Motion Detection Using CCTV Camera with Python**" and gain practical knowledge in Python programming and computer vision.

I extend my gratitude to my parents, friends, and classmates for their constant support, encouragement, and cooperation during the completion of this project.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of this mini project.

Abstract

Motion detection is one of the most important applications of computer vision and is widely used in security and surveillance systems. The main objective of this project is to detect moving objects in front of a CCTV camera or webcam and provide real-time monitoring using Python and OpenCV.

The system captures live video from a camera and continuously compares consecutive video frames. When a significant difference between the frames is detected, the system identifies it as motion and highlights the moving object by drawing a bounding box around it. This helps users easily recognize any movement occurring within the camera's field of view.

The project is developed using the Python programming language with the OpenCV library for image processing and motion detection. Additional libraries such as NumPy are used for efficient data processing. The system is simple, cost-effective, and can be implemented using a standard webcam or CCTV camera without requiring expensive hardware.

This project demonstrates the practical application of computer vision techniques in real-time surveillance. It can be used in homes, offices, banks, schools, hospitals, warehouses, parking areas, and other locations where continuous monitoring is required. The project also provides a strong foundation for future enhancements, such as artificial intelligence-based human detection, face recognition, object classification, intrusion detection, automatic video recording, and cloud-based monitoring.

Overall, this project offers an efficient and reliable solution for detecting motion in real time, improving security, reducing manual monitoring, and supporting the development of intelligent surveillance systems.

Chapter 1 – Introduction

1.1 Introduction

Motion detection is an important technology in the field of computer vision and video surveillance. It is used to identify movement within the viewing area of a camera by analyzing consecutive video frames. Motion detection systems are widely used in homes, offices, banks, schools, hospitals, shopping malls, parking areas, and industries to improve security and reduce the need for continuous human monitoring.

The project **"Motion Detection Using CCTV Camera with Python"** is designed to detect moving objects in real time using a CCTV camera or webcam. The system captures live video from the camera and compares consecutive frames to identify changes. If a significant change is detected, the system recognizes it as motion and highlights the moving object with a rectangular bounding box. This allows users to easily identify any movement occurring in the monitored area.

This project is implemented using the Python programming language and the OpenCV library, which provides powerful tools for image processing and computer vision. NumPy is also used to perform efficient mathematical operations on image data. The system is simple, cost-effective, and easy to implement, making it suitable for educational purposes as well as basic security applications.

Motion detection plays a vital role in modern surveillance systems by reducing unnecessary video recording and enabling quick responses to suspicious activities. Instead of continuously monitoring video footage, the system automatically detects movement and alerts the user whenever motion occurs. This saves storage space, reduces monitoring effort, and improves overall security.

The project demonstrates the practical application of image processing techniques in real-time surveillance. It also serves as a foundation for advanced technologies such as artificial intelligence, machine learning, face recognition, object detection, intrusion detection, and smart surveillance systems.

1.2 Background of the Project

Security has become one of the most important concerns in today's world. Traditional surveillance systems require continuous monitoring by security personnel, which is time-consuming and may lead to human errors. Motion detection technology helps overcome these limitations by automatically detecting movement within the camera's field of view.

With the advancement of computer vision and image processing, motion detection systems have become faster, more accurate, and more affordable. Python and OpenCV provide an excellent platform for developing such intelligent surveillance applications.

1.3 Need for the Project

The need for this project arises from the increasing demand for automated security and surveillance systems. Manual monitoring of CCTV footage is difficult and requires constant attention. A motion detection system automatically identifies movement and helps users monitor important events efficiently.

This project offers several benefits:

- Improves security by detecting unauthorized movement.
- Reduces the need for continuous human monitoring.
- Saves storage by recording only when motion is detected.
- Provides real-time monitoring and quick response.
- Can be implemented at a low cost using a webcam or CCTV camera.

1.4 Scope of the Project

The scope of this project includes real-time motion detection using a webcam or CCTV camera. The system detects moving objects, highlights them using bounding boxes, and can be further enhanced to save images, record videos, send alerts, or integrate artificial intelligence features.

Future improvements may include:

- Human detection using Artificial Intelligence.
- Face recognition for authorized users.
- Mobile notifications and email alerts.

- Cloud storage for recorded videos.
- Integration with IoT-based smart security systems.

1.5 Summary

This chapter introduced the concept of motion detection, explained its importance in surveillance systems, and discussed the objectives, need, and scope of the project. The following chapters provide a detailed explanation of the literature survey, methodology, system design, implementation, results, and future enhancements of the project.

Chapter 2 – Literature Survey

2.1 Introduction

A literature survey is an important part of any project as it provides an overview of previous research and existing technologies related to the project. Motion detection has been widely studied in the fields of computer vision, image processing, and video surveillance. Many researchers have developed different techniques to detect moving objects accurately and efficiently.

The purpose of this literature survey is to understand the methods, tools, and technologies used in motion detection systems and identify the most suitable approach for this project.

2.2 Motion Detection

Motion detection is the process of identifying movement by comparing consecutive video frames captured by a camera. If a significant difference is found between the frames, the system considers it as motion. This technique is commonly used in CCTV surveillance systems to monitor activities in real time.

Motion detection helps reduce manual monitoring and enables automatic detection of suspicious activities. It is widely used in homes, offices, industries, hospitals, banks, shopping malls, and public places.

2.3 Existing Systems

Traditional surveillance systems mainly record video continuously without analyzing the content. Security personnel must monitor the footage manually, which is time-consuming and may result in missed events due to human error.

Some existing systems use basic sensors such as Passive Infrared (PIR) sensors to detect movement. These sensors can detect heat changes but cannot identify the exact location or shape of moving objects.

Modern surveillance systems use computer vision techniques to analyze video frames and detect moving objects automatically. These systems are more accurate and efficient than traditional methods.

2.4 Technologies Used

Python

Python is a high-level programming language known for its simplicity and readability. It provides powerful libraries for image processing and machine learning, making it an excellent choice for developing motion detection systems.

OpenCV

OpenCV (Open Source Computer Vision Library) is one of the most popular libraries for image processing and computer vision. It provides functions for video capture, image filtering, contour detection, frame comparison, and object tracking.

NumPy

NumPy is a scientific computing library used for performing mathematical operations on arrays and images. It improves the speed and efficiency of image processing tasks.

Webcam/CCTV Camera

A webcam or CCTV camera is used to capture live video. The captured frames are processed continuously to detect movement within the monitored area.

2.5 Motion Detection Techniques

Several techniques are available for motion detection, including:

Frame Differencing:

This method compares two consecutive video frames. If there is a significant difference between them, motion is detected. It is simple, fast, and suitable for real-time applications.

Background Subtraction:

In this method, the current video frame is compared with a reference background image. Any changes from the background are identified as moving objects.

Optical Flow:

This technique calculates the direction and speed of moving objects between consecutive frames. It provides accurate motion analysis but requires more computational power.

For this project, the **Frame Differencing** technique is selected because it is easy to implement, computationally efficient, and suitable for beginners.

2.6 Applications of Motion Detection

Motion detection technology is widely used in various fields, including:

- Home security systems
- Office surveillance
- Banks and ATMs
- Hospitals
- Educational institutions
- Shopping malls
- Parking areas
- Industrial monitoring
- Smart city surveillance
- Traffic monitoring

2.7 Research Gap

Although many surveillance systems are available, some are expensive and require specialized hardware. Small organizations and educational institutions often need a low-cost and easy-to-use solution.

This project addresses this need by developing a simple motion detection system using Python, OpenCV, and a standard webcam or CCTV camera. The system is affordable, easy to implement, and suitable for learning as well as basic security applications.

2.8 Summary

The literature survey shows that motion detection is an essential component of modern surveillance systems. Various techniques have been developed over the years, with frame differencing being one of the simplest and most effective methods for real-time motion

detection. Based on the review of existing systems and technologies, Python and OpenCV were selected for developing this project due to their flexibility, ease of use, and strong support for computer vision applications.

Chapter 3 – Problem Statement

3.1 Introduction

Security and surveillance have become essential in today's world due to the increasing need to protect people, property, and valuable assets. Traditional CCTV systems continuously record video, requiring security personnel to monitor the footage at all times. Continuous monitoring is time-consuming, expensive, and prone to human error, making it difficult to identify important events efficiently.

To overcome these challenges, an automated motion detection system is required. Such a system can monitor live video, detect movement automatically, and notify users whenever motion is identified. This reduces manual effort and improves the overall efficiency of surveillance systems.

3.2 Problem Statement

The main problem addressed in this project is the inability of traditional surveillance systems to automatically detect motion and alert users in real time. Continuous video recording consumes large amounts of storage space, while manual monitoring can result in missed incidents due to lack of attention or fatigue.

This project aims to develop a simple, reliable, and cost-effective motion detection system using Python and OpenCV. The system captures live video through a CCTV camera or webcam, compares consecutive frames, detects moving objects, and highlights the detected motion with a bounding box. This helps users quickly identify suspicious or important activities without continuously watching the video feed.

3.3 Challenges in Existing Systems

The following problems are commonly found in traditional surveillance systems:

- Continuous recording consumes a large amount of storage space.
- Manual monitoring requires constant human attention.
- Important events may be missed due to human error or fatigue.
- Many advanced surveillance systems are expensive to install and maintain.

- Traditional systems cannot automatically identify moving objects without additional software.

3.4 Proposed Solution

To solve these problems, this project proposes an automated motion detection system that uses computer vision techniques. The system continuously captures video frames and compares them to detect movement. When motion is detected, the moving object is highlighted, allowing users to identify activity instantly.

The proposed system is:

- Simple to develop and use.
- Low-cost and suitable for educational purposes.
- Capable of detecting motion in real time.
- Compatible with a webcam or CCTV camera.
- Easy to enhance with features such as video recording, image capture, alerts, and AI-based object detection.

3.5 Summary

The problem statement highlights the limitations of traditional surveillance systems and explains the need for an automated motion detection solution. By using Python and OpenCV, the proposed system provides an efficient, reliable, and affordable method for detecting motion in real time, making it suitable for homes, offices, educational institutions, and small businesses.

Chapter 4 – Objectives

4.1 Introduction

The objectives of this project define the goals and expected outcomes of developing a motion detection system using a CCTV camera and Python. The project aims to create a simple, efficient, and cost-effective surveillance solution capable of detecting movement in real time. By using computer vision techniques, the system automatically identifies motion and reduces the need for continuous manual monitoring.

4.2 Main Objective

The main objective of this project is to develop a real-time motion detection system using Python and OpenCV that detects moving objects through a CCTV camera or webcam and highlights them for effective surveillance.

4.3 Specific Objectives

The specific objectives of this project are:

- To capture live video using a CCTV camera or webcam.
- To compare consecutive video frames for motion detection.
- To identify moving objects accurately in real time.
- To draw a bounding box around detected moving objects.
- To reduce the need for continuous manual monitoring.
- To develop a low-cost and user-friendly surveillance system.
- To improve security in homes, offices, schools, and other public places.
- To understand the practical application of computer vision and image processing.
- To implement the project using Python, OpenCV, and NumPy.
- To create a system that can be enhanced with future technologies such as Artificial Intelligence and Machine Learning.

4.4 Project Goals

The project is designed to achieve the following goals:

- Detect motion quickly and accurately.
- Monitor live video continuously.
- Reduce unnecessary video recording and storage usage.
- Provide an easy-to-use surveillance solution.
- Demonstrate the practical implementation of motion detection using computer vision.
- Build a foundation for advanced smart surveillance systems.

4.5 Expected Outcomes

After completing this project, the expected outcomes are:

- Successful real-time motion detection using a webcam or CCTV camera.
- Automatic identification of moving objects.
- Display of detected motion with a rectangular bounding box.
- Better understanding of Python programming and OpenCV.
- A reliable and cost-effective surveillance system suitable for educational and basic security purposes.

4.6 Summary

The objectives of this project focus on developing a simple, efficient, and reliable motion detection system that improves security through automated surveillance. The project also provides practical knowledge of Python programming, image processing, and computer vision while creating a foundation for future enhancements using Artificial Intelligence and Machine Learning.

Chapter 5 – System Requirements

5.1 Introduction

System requirements define the hardware and software needed to develop and run the Motion Detection Using CCTV Camera project. Proper hardware and software ensure that the application works efficiently and detects motion in real time.

5.2 Hardware Requirements

The following hardware components are required for this project:

Component	Specification
Processor	Intel Core i3 or higher
RAM	4 GB or above (8 GB recommended)
Storage	Minimum 100 GB free space
Camera	Webcam or CCTV Camera
Monitor	Standard Display
Keyboard & Mouse	Required for system operation

5.3 Software Requirements

The following software is used to develop and execute the project:

Software	Purpose
Operating System	Windows 10/11
Programming Language	Python 3.x
IDE	Visual Studio Code (VS Code)
Library	OpenCV
Library	NumPy

5.4 Python Libraries Used

OpenCV (cv2)

OpenCV is an open-source computer vision library used for capturing video, processing images, detecting motion, and drawing bounding boxes around moving objects.

NumPy

NumPy is a numerical computing library that performs fast mathematical operations on image data and supports efficient frame processing.

Datetime

The `datetime` module is used to record the current date and time whenever motion is detected. This helps in maintaining logs of detected events.

Winsound (Windows)

The `winsound` module is used to generate an alert sound whenever motion is detected. It is available only on Windows operating systems.

5.5 Functional Requirements

The system should be able to:

- Capture live video from a webcam or CCTV camera.
- Compare consecutive video frames.
- Detect moving objects accurately.
- Display a bounding box around detected motion.
- Show live video output continuously.
- Exit the application when the user presses the specified key.

5.6 Non-Functional Requirements

The system should satisfy the following non-functional requirements:

- Easy to use.

- Fast and responsive.
- Reliable during continuous operation.
- Low-cost implementation.
- Easy to maintain and upgrade.
- Compatible with standard Windows computers.

5.7 Summary

The system requirements define the hardware and software needed to successfully develop and execute the motion detection project. Using Python, OpenCV, and NumPy with a standard webcam or CCTV camera provides a simple, efficient, and cost-effective solution for real-time motion detection.

Chapter 6 – Methodology

6.1 Introduction

Methodology describes the step-by-step process used to develop the **Motion Detection Using CCTV Camera with Python** project. It explains how the system captures video, processes each frame, detects motion, and displays the results in real time. The project uses computer vision techniques with Python and OpenCV to provide an efficient and accurate motion detection system.

6.2 Methodology Flow

The motion detection process follows these steps:

1. Start the system.
2. Initialize the webcam or CCTV camera.
3. Capture live video frames continuously.
4. Convert the frames into grayscale images.
5. Apply Gaussian Blur to reduce image noise.
6. Compare two consecutive frames.
7. Calculate the difference between the frames.
8. Apply thresholding to identify moving regions.
9. Detect contours of the moving objects.
10. Draw a bounding box around the detected object.
11. Display the processed video output.
12. Repeat the process until the user exits the application.

6.3 Working Procedure

Step 1: Video Capture

The system starts by connecting to a webcam or CCTV camera. It continuously captures live video frames using the OpenCV `VideoCapture()` function.

Step 2: Frame Pre-processing

Each captured frame is converted into a grayscale image because grayscale processing is faster than color image processing. A Gaussian Blur filter is applied to remove noise and improve detection accuracy.

Step 3: Frame Comparison

The system compares two consecutive frames. If there is a significant difference between them, it indicates that motion has occurred.

Step 4: Thresholding

A threshold is applied to the difference image to separate moving objects from the background. Small unwanted changes are removed to improve accuracy.

Step 5: Contour Detection

OpenCV detects the boundaries (contours) of the moving objects. Small contours caused by noise are ignored.

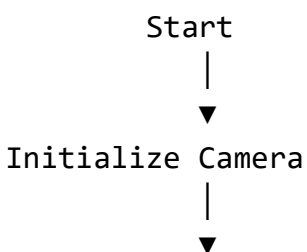
Step 6: Motion Detection

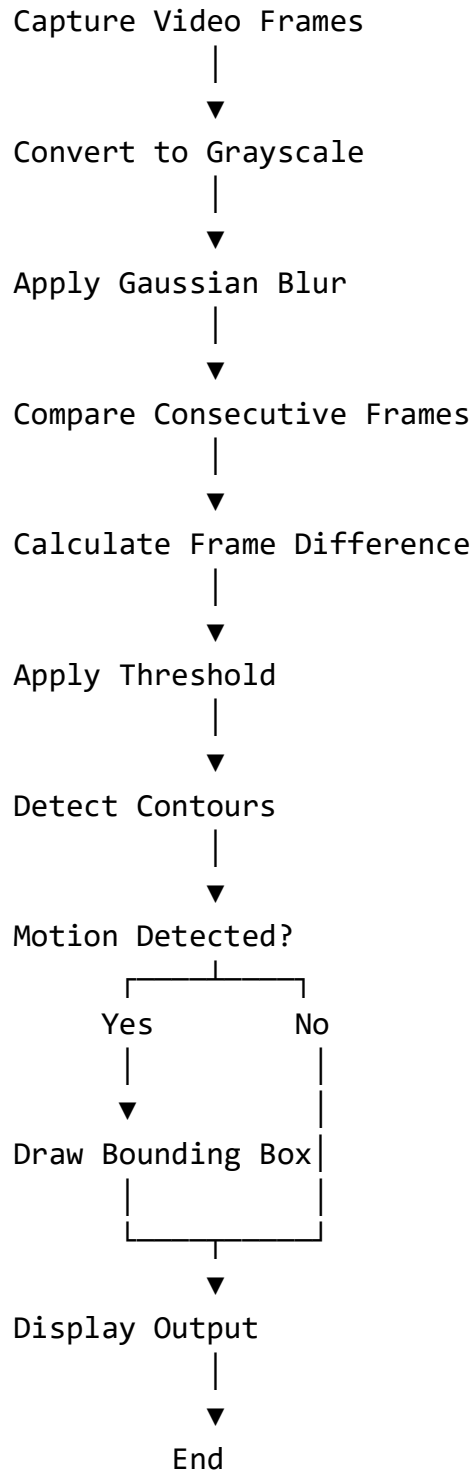
When the contour area is larger than a predefined value, the system identifies it as valid motion.

Step 7: Display Output

A green rectangular bounding box is drawn around the detected moving object. The live video is displayed on the screen until the user closes the application.

6.4 Methodology Diagram





6.5 Advantages of the Methodology

- Simple and easy to implement.

- Detects motion in real time.
- Uses low-cost hardware.
- Fast image processing with OpenCV.
- Can be upgraded with AI and Machine Learning features.

6.6 Summary

The methodology explains the complete workflow of the motion detection system. The system captures live video, processes the frames, detects movement using frame differencing and contour detection, and displays the moving objects with bounding boxes. This approach provides a simple, reliable, and efficient solution for real-time surveillance applications.

Chapter 7 – System Design

7.1 Introduction

System design is an important phase in the development of the **Motion Detection Using CCTV Camera with Python** project. It defines the overall structure, components, and workflow of the system. The design explains how the webcam or CCTV camera captures video, how the software processes the video frames, and how the detected motion is displayed to the user.

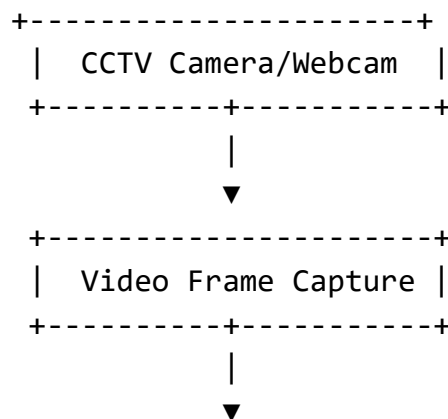
The system is designed to operate in real time by continuously capturing video frames, processing them using computer vision techniques, detecting movement, and displaying the output with highlighted moving objects.

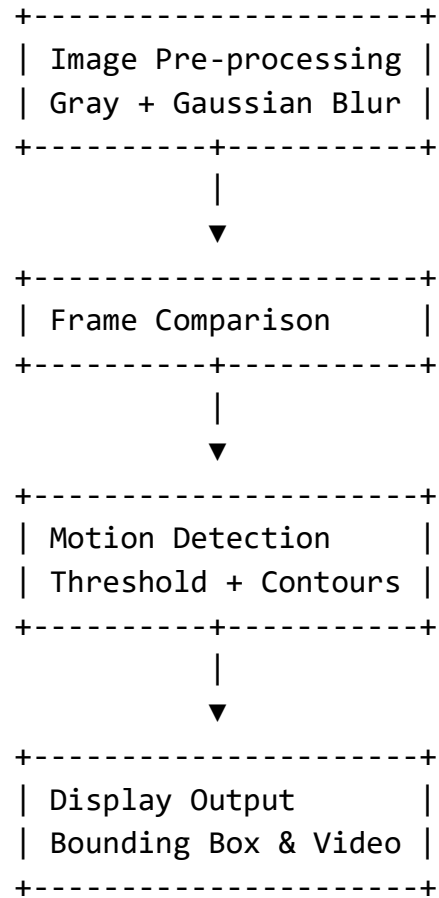
7.2 System Architecture

The system consists of the following main components:

- **Input Device:** Webcam or CCTV Camera
- **Processing Unit:** Python with OpenCV and NumPy
- **Motion Detection Module:** Frame Differencing and Contour Detection
- **Output Module:** Live video with detected motion highlighted using bounding boxes

Architecture Diagram





7.3 Input Design

The input to the system is live video captured through a webcam or CCTV camera. The camera continuously captures frames, which are forwarded to the motion detection algorithm for processing.

Input Components

- Webcam or CCTV Camera
- Live Video Stream
- Continuous Frame Capture

7.4 Process Design

The processing module is responsible for detecting motion. It performs the following operations:

1. Capture consecutive video frames.
2. Convert frames into grayscale images.
3. Apply Gaussian Blur to reduce image noise.
4. Calculate the difference between consecutive frames.
5. Apply thresholding to separate moving objects.
6. Detect contours.
7. Ignore small contours caused by noise.
8. Draw bounding boxes around detected moving objects.

This process is repeated continuously until the application is closed.

7.5 Output Design

The output of the system is displayed in a live video window.

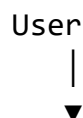
When motion is detected:

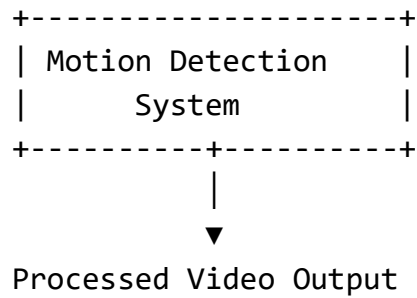
- A green rectangular bounding box appears around the moving object.
- Live video continues without interruption.
- Motion is monitored continuously in real time.

If no motion is detected, the system simply displays the live camera feed.

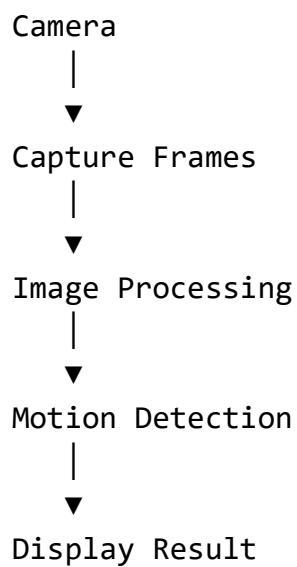
7.6 Data Flow Diagram (DFD)

Level-0 DFD





Level-1 DFD



7.7 Module Design

The project is divided into the following modules:

Module 1: Video Capture Module

- Captures live video from the webcam or CCTV camera.
- Sends video frames to the processing module.

Module 2: Image Processing Module

- Converts frames into grayscale.
- Removes image noise using Gaussian Blur.
- Improves image quality for accurate detection.

Module 3: Motion Detection Module

- Compares consecutive frames.
- Calculates frame differences.
- Detects moving objects using contour detection.

Module 4: Output Module

- Draws bounding boxes around moving objects.
- Displays the processed video.
- Continues monitoring until the program is terminated.

7.8 System Workflow

The overall workflow of the system is as follows:

1. Start the application.
2. Initialize the webcam or CCTV camera.
3. Capture live video.
4. Process each frame.
5. Detect motion.
6. Highlight moving objects.
7. Display the output.
8. Repeat until the user exits.

7.9 Summary

The system design describes the architecture, modules, input, processing, output, and data flow of the motion detection system. The modular design makes the system simple, efficient, and easy to understand. It also provides flexibility for future enhancements such as AI-based human detection, face recognition, automatic video recording, and cloud-based surveillance.

Chapter 8 – Algorithm

8.1 Introduction

An algorithm is a step-by-step procedure used to solve a problem or perform a specific task. In this project, the algorithm explains how the motion detection system captures live video, processes each frame, detects movement, and displays the results. The algorithm is designed to be simple, efficient, and suitable for real-time surveillance applications.

8.2 Motion Detection Algorithm

The following algorithm is used in the **Motion Detection Using CCTV Camera with Python** project.

Algorithm Steps

Step 1: Start the program.

Step 2: Import the required Python libraries such as OpenCV (cv2), NumPy, Datetime, and Winsound (optional).

Step 3: Initialize the webcam or CCTV camera using the `VideoCapture()` function.

Step 4: Capture two consecutive video frames from the camera.

Step 5: Convert both frames into grayscale images.

Step 6: Apply Gaussian Blur to both grayscale images to reduce noise.

Step 7: Calculate the absolute difference between the two frames.

Step 8: Apply thresholding to convert the difference image into a binary image.

Step 9: Dilate the binary image to fill small gaps and improve object detection.

Step 10: Detect contours from the processed image.

Step 11: Check the contour area.

- If the contour area is smaller than the predefined threshold, ignore it.
- Otherwise, consider it as a moving object.

Step 12: Draw a green bounding box around the detected moving object.

Step 13: Display the processed video frame on the screen.

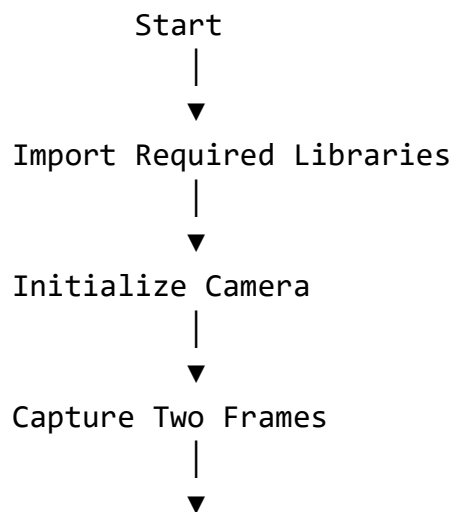
Step 14: Update the previous frame and capture the next frame.

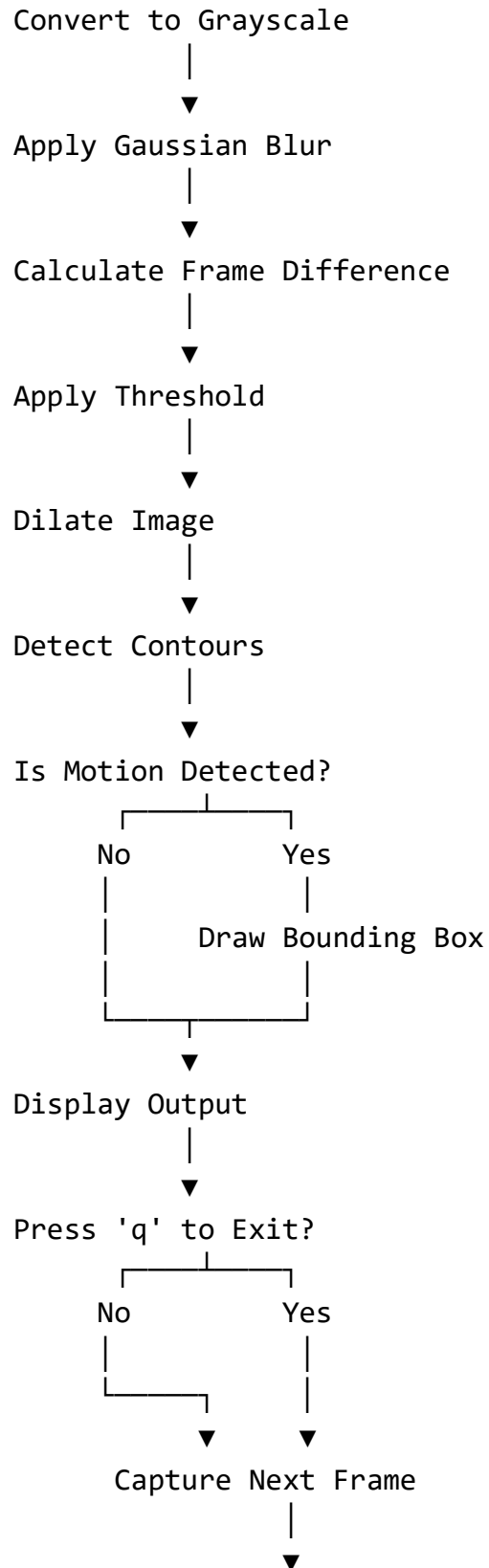
Step 15: Repeat the above steps until the user presses the 'q' key to exit the program.

Step 16: Release the camera and close all OpenCV windows.

Step 17: End the program.

8.3 Algorithm Flow





End

8.4 Algorithm Advantages

The proposed algorithm offers several advantages:

- Easy to understand and implement.
- Detects motion in real time.
- Requires only a webcam or CCTV camera.
- Provides fast processing using OpenCV.
- Reduces unnecessary video recording.
- Low-cost solution for surveillance applications.
- Can be enhanced with Artificial Intelligence and Machine Learning techniques.

8.5 Time Complexity

The algorithm processes each video frame sequentially. The overall processing time depends on the number of pixels in each frame. Since each frame is processed once, the algorithm performs efficiently and is suitable for real-time applications on standard computers.

8.6 Summary

The algorithm provides a systematic procedure for detecting motion using frame differencing and contour detection. By continuously comparing consecutive frames, the system accurately identifies moving objects and highlights them using bounding boxes. The algorithm is efficient, reliable, and forms the core of the motion detection system developed in this project.

Chapter 9 – Implementation (Python Code)

9.1 Introduction

Implementation is the phase in which the designed system is converted into a working application. In this project, the motion detection system is implemented using the Python programming language and the OpenCV library. The program captures live video from a webcam or CCTV camera, processes each frame, detects motion, and displays the moving objects with a bounding box in real time.

9.2 Tools and Technologies Used

The following tools and technologies were used for implementing the project:

Tool/Technology	Purpose
Python 3.x	Programming Language
OpenCV	Image Processing and Motion Detection
NumPy	Numerical and Array Operations
Visual Studio Code	Code Editor
Webcam/CCTV Camera	Video Input Device
Windows 10/11	Operating System

9.3 Python Source Code

```
import cv2
import winsound
from datetime import datetime

# Start video capture
cap = cv2.VideoCapture(0)

# Read initial frames
```

```

ret, frame1 = cap.read()
ret, frame2 = cap.read()

while cap.isOpened():

    # Find the difference between two frames
    diff = cv2.absdiff(frame1, frame2)

    # Convert to grayscale
    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)

    # Apply Gaussian Blur
    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    # Apply threshold
    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)

    # Dilate image
    dilated = cv2.dilate(thresh, None, iterations=3)

    # Find contours
    contours, _ = cv2.findContours(
        dilated,
        cv2.RETR_TREE,
        cv2.CHAIN_APPROX_SIMPLE
    )

    motion = False

    for contour in contours:

        if cv2.contourArea(contour) < 1000:
            continue

        motion = True

        (x, y, w, h) = cv2.boundingRect(contour)

        # Draw rectangle

```



```

        cv2.rectangle(frame1, (x, y), (x + w, y + h),
                       (0, 255, 0), 2)

    if motion:
        winsound.Beep(1000, 200)
        current_time = datetime.now().strftime("%d-%m-%Y %H:%M:%S")
        cv2.putText(frame1,
                    "Motion Detected: " + current_time,
                    (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.7,
                    (0, 0, 255),
                    2)

# Display output
cv2.imshow("Motion Detection", frame1)

# Update frames
frame1 = frame2
ret, frame2 = cap.read()

# Exit on pressing q
if cv2.waitKey(10) & 0xFF == ord('q'):
    break

# Release camera
cap.release()
cv2.destroyAllWindows()

```

9.4 Code Explanation

The implementation consists of the following steps:

Step 1: Import Libraries

The required Python libraries such as **OpenCV**, **Winsound**, and **Datetime** are imported.

Step 2: Capture Live Video

The webcam or CCTV camera is initialized using the `VideoCapture()` function.

Step 3: Read Video Frames

Two consecutive frames are captured continuously from the camera.

Step 4: Calculate Frame Difference

The system compares the two frames using the `absdiff()` function to identify changes.

Step 5: Convert to Grayscale

The difference image is converted to grayscale for faster image processing.

Step 6: Apply Gaussian Blur

Noise is reduced using Gaussian Blur, improving the accuracy of motion detection.

Step 7: Thresholding

Thresholding converts the grayscale image into a binary image where moving objects become clearly visible.

Step 8: Dilate the Image

Image dilation fills small gaps in the detected regions, making contour detection more accurate.

Step 9: Detect Contours

Contours are detected to identify the boundaries of moving objects.

Step 10: Draw Bounding Box

A green rectangular box is drawn around the detected moving object.

Step 11: Display Alert

When motion is detected:

- A beep sound is generated.
- The current date and time are displayed on the screen.

Step 12: Display Output

The processed video with the bounding box is displayed continuously.

Step 13: Exit Program

The program terminates when the user presses the 'q' key.

9.5 Implementation Results

The implemented system successfully:

- Captures live video from the webcam or CCTV camera.
- Detects moving objects in real time.
- Draws a green bounding box around detected motion.
- Displays the current date and time during motion detection.
- Generates an alert sound when movement is detected.
- Provides continuous surveillance until the application is closed.

9.6 Summary

The implementation chapter demonstrates the successful development of the motion detection system using Python and OpenCV. The project effectively detects motion by comparing consecutive video frames and highlights moving objects with bounding boxes. The implemented system is simple, efficient, reliable, and suitable for real-time surveillance applications. It also provides a strong foundation for future enhancements

Such as AI-based object detection, face recognition, automatic recording, and cloud-based monitoring.

Chapter 10 – Results and Output Screenshots

10.1 Introduction

This chapter presents the results obtained after implementing the **Motion Detection Using CCTV Camera with Python** project. The application was tested using a webcam/CCTV camera under different conditions. The system successfully detected moving objects in real time by comparing consecutive video frames. Whenever motion was detected, a green bounding box was displayed around the moving object, and the current date and time were shown on the screen.

The experimental results demonstrate that the system performs efficiently in detecting motion and can be used for basic surveillance applications.

10.2 Test Environment

The project was tested using the following configuration:

Component	Specification
Operating System	Windows 10/11
Programming Language	Python 3.x
IDE	Visual Studio Code
Libraries	OpenCV, NumPy
Camera	Webcam/CCTV Camera
Processor	Intel Core i3 or Higher
RAM	4 GB or Above

10.3 Output – Program Execution

Output 1: Program Started Successfully

The application starts successfully and opens the webcam or CCTV camera. The live video stream is displayed in a window, and the system continuously monitors the environment for movement.

(Insert Screenshot 10.1 – Live Camera Window Here)

Figure 10.1: Live video captured from the webcam/CCTV camera.

Output 2: Motion Detected

When a person or object moves in front of the camera, the system detects the movement by comparing consecutive video frames. A green rectangular bounding box is drawn around the moving object, indicating successful motion detection.

(Insert Screenshot 10.2 – Motion Detection with Bounding Box Here)

Figure 10.2: Motion detected with a green bounding box around the moving object.

Output 3: Date and Time Display

After detecting motion, the system displays the current date and time on the video frame. This helps users identify when the motion occurred and can be useful for surveillance records.

(Insert Screenshot 10.3 – Motion Detection with Date and Time Here)

Figure 10.3: Motion detection displaying the current date and time.

Output 4: Alert Sound (Optional)

Whenever motion is detected, the system generates a beep sound using the Winsound library to notify the user immediately.

Result: Alert sound generated successfully when motion was detected.

10.4 Performance Analysis

The project was tested under different conditions, and the following observations were made:

Test Case	Expected Result	Actual Result	Status
Camera Initialization	Camera should open	Camera opened successfully	Passed
Live Video Capture	Display live video	Video displayed successfully	Passed
Motion Detection	Detect moving objects	Motion detected accurately	Passed
Bounding Box	Highlight moving object	Bounding box displayed	Passed
Date and Time	Display timestamp	Timestamp displayed	Passed
Exit Program	Close on pressing 'q'	Program closed successfully	Passed

10.5 Observations

The following observations were made during project testing:

- The system successfully detected moving objects in real time.
- Motion detection worked efficiently under normal lighting conditions.
- Small image noise was reduced using Gaussian Blur.
- The application processed video frames smoothly.
- The system provided accurate results for basic surveillance applications.
- Performance depended on lighting conditions and camera quality.

10.6 Result Summary

The project achieved all the planned objectives successfully. The developed application:

- Captured live video from a webcam or CCTV camera.
- Detected motion accurately.
- Displayed a green bounding box around moving objects.
- Showed the current date and time during motion detection.
- Generated an alert sound (optional).
- Worked efficiently in real-time surveillance.

The project proved to be a reliable, cost-effective, and easy-to-use motion detection system suitable for educational purposes and basic security applications.

10.7 Summary

The results obtained from the implementation confirm that the proposed motion detection system functions successfully. The system accurately detects movement, highlights moving objects, and displays the output in real time. The project demonstrates the practical application of Python and OpenCV in developing intelligent surveillance systems and provides a strong foundation for future enhancements using Artificial Intelligence and Machine Learning.

Chapter 11 – Advantages

11.1 Introduction

Every project has certain benefits that make it useful in real-world applications. The **Motion Detection Using CCTV Camera with Python** project offers several advantages in the field of security and surveillance. It helps in automatically detecting movement, reducing manual monitoring, and improving the overall efficiency of surveillance systems.

11.2 Advantages of the Proposed System

The major advantages of the proposed motion detection system are:

1. Real-Time Motion Detection

The system detects motion instantly by continuously analyzing live video captured from a webcam or CCTV camera. This enables quick identification of any movement in the monitored area.

2. Improved Security

The project enhances security by automatically identifying unauthorized movement. It helps users monitor homes, offices, schools, banks, and other locations more effectively.

3. Low-Cost Solution

The system can be implemented using a standard webcam or CCTV camera and open-source software such as Python and OpenCV. This makes it an affordable surveillance solution.

4. Easy to Use

The application has a simple design and is easy to operate. Users with basic computer knowledge can run the program without difficulty.

5. Reduces Manual Monitoring

The system automatically detects motion, reducing the need for continuous observation of CCTV footage by security personnel.

6. Fast Processing

OpenCV processes video frames efficiently, allowing the system to detect motion in real time with minimal delay.

7. Efficient Use of Storage

The system can be enhanced to record video only when motion is detected, reducing unnecessary recordings and saving storage space.

8. Accurate Motion Detection

By comparing consecutive frames and using contour detection, the system accurately identifies moving objects while reducing false detections caused by minor image changes.

9. Easy Maintenance

The software is simple to maintain and update. Additional features can be added without redesigning the entire system.

10. Scalable Design

The project can be expanded to support multiple cameras, cloud storage, mobile notifications, and advanced AI-based surveillance features.

11.3 Educational Advantages

This project also provides valuable learning opportunities for students by helping them understand:

- Python programming.
- Computer Vision concepts.

- OpenCV library functions.
- Image processing techniques.
- Motion detection algorithms.
- Real-time video processing.
- Practical software development.

11.4 Practical Applications

The proposed system can be used in various places, including:

- Homes
- Offices
- Banks
- Schools and Colleges
- Hospitals
- Shopping Malls
- Parking Areas
- Warehouses
- Industrial Premises
- Public Surveillance Systems

11.5 Summary

The **Motion Detection Using CCTV Camera with Python** project provides a reliable, efficient, and economical solution for real-time surveillance. It improves security, reduces manual effort, and demonstrates the practical application of computer vision techniques. The system is easy to implement, simple to maintain, and can be enhanced with Artificial Intelligence and Machine Learning for advanced surveillance applications in the future.

Chapter 12 – Limitations

12.1 Introduction

Every software system has certain limitations that affect its performance under specific conditions. Although the **Motion Detection Using CCTV Camera with Python** project provides an efficient and low-cost solution for detecting motion, it also has some limitations. Understanding these limitations helps identify areas for future improvement and enhances the overall performance of the system.

12.2 Limitations of the Proposed System

1. Sensitive to Lighting Conditions

The system may produce inaccurate results when there are sudden changes in lighting, such as switching lights on or off, sunlight variations, or shadows.

2. Cannot Identify Objects

The project detects only motion and cannot determine whether the moving object is a person, animal, vehicle, or any other object.

3. Limited Camera Coverage

The system can detect motion only within the camera's field of view. Any movement outside the camera's range cannot be detected.

4. False Motion Detection

Environmental factors such as moving tree branches, rain, insects, or camera vibrations may cause false motion detection.

5. Depends on Camera Quality

The accuracy of motion detection depends on the resolution and quality of the webcam or CCTV camera. Low-quality cameras may reduce detection accuracy.

6. No Remote Monitoring

The current system operates on a local computer and does not support remote monitoring through mobile devices or the internet.

7. No Automatic Notifications

The system does not send SMS, email, or mobile notifications when motion is detected. Users must monitor the application directly.

8. Limited AI Capabilities

The project uses traditional image processing techniques and does not include Artificial Intelligence or Machine Learning for object recognition or behavior analysis.

9. Storage Feature Not Included

The current implementation does not automatically save detected images or record video footage unless additional functionality is added.

10. Platform Dependency

The alert sound in the project uses the Windows winsound library, which works only on Windows operating systems. Alternative methods are required for Linux or macOS.

12.3 Possible Improvements

The above limitations can be addressed by implementing the following enhancements:

- Integrate Artificial Intelligence for human and object detection.
- Add face recognition for identifying authorized persons.
- Send real-time email or mobile notifications when motion is detected.

- Enable automatic image capture and video recording.
- Store recordings securely using cloud storage.
- Support multiple CCTV cameras simultaneously.
- Improve accuracy under different lighting conditions.
- Develop a mobile application for remote monitoring and control.

12.4 Summary

Although the proposed motion detection system has some limitations, it successfully achieves its primary objective of detecting motion in real time. Most of the limitations can be overcome by incorporating advanced technologies such as Artificial Intelligence, Machine Learning, cloud computing, and IoT. Therefore, the project provides a strong foundation for developing more intelligent and advanced surveillance systems in the future.

Chapter 13 – Applications

13.1 Introduction

Motion detection technology has become an essential part of modern security and surveillance systems. It is widely used in various sectors to monitor activities, detect unauthorized movement, and improve safety. The **Motion Detection Using CCTV Camera with Python** project demonstrates how computer vision can be used to automate surveillance and reduce the need for continuous manual monitoring.

13.2 Applications of the Proposed System

The motion detection system can be used in the following areas:

1. Home Security

The system helps homeowners monitor their property by detecting unauthorized movement around entrances, rooms, garages, or backyards. It enhances home security by providing real-time motion detection.

2. Office Surveillance

Organizations can use the system to monitor offices, workspaces, and restricted areas. It helps prevent unauthorized access and protects valuable equipment and confidential information.

3. Banks and ATMs

Banks and ATM centers require continuous surveillance to ensure the safety of customers and assets. Motion detection can help identify suspicious activities and improve overall security.

4. Schools and Colleges

Educational institutions can use the system to monitor classrooms, laboratories, libraries, corridors, and campus entrances. It helps improve student safety and protect institutional property.

5. Hospitals

Hospitals can install motion detection systems in restricted areas such as medicine storage rooms, laboratories, and emergency exits to improve security and monitor unauthorized access.

6. Shopping Malls and Retail Stores

Retail stores and shopping malls can use motion detection to monitor customer movement, prevent theft, and improve the safety of employees and visitors.

7. Parking Areas

The system can monitor parking lots and detect vehicle or human movement, helping improve vehicle security and reduce unauthorized access.

8. Warehouses and Industries

Industries and warehouses can use motion detection to monitor storage areas, machinery sections, and production units, reducing the risk of theft and unauthorized entry.

9. Public Places

Motion detection systems can be installed in railway stations, airports, bus terminals, parks, and government buildings to enhance public safety and monitor crowd movement.

10. Smart Cities

Modern smart city projects use intelligent surveillance systems with motion detection to improve traffic management, public safety, and urban security.

13.3 Educational Applications

This project is also useful for educational purposes. Students can use it to learn:

- Python Programming
- OpenCV Library
- Computer Vision
- Image Processing
- Real-Time Video Processing
- Artificial Intelligence Basics
- Machine Learning Applications in Surveillance

13.4 Industrial Applications

Many industries use motion detection technology for:

- Employee safety monitoring
- Automated surveillance
- Factory security
- Inventory protection
- Equipment monitoring
- Restricted area access control

13.5 Summary

The **Motion Detection Using CCTV Camera with Python** project has a wide range of applications in security, surveillance, education, healthcare, industries, banking, retail, transportation, and smart cities. Its low cost, ease of implementation, and real-time performance make it a practical solution for various monitoring applications. With future enhancements such as Artificial Intelligence, Machine Learning, cloud storage, and IoT integration, the system can become even more intelligent and efficient.

Chapter 14 – Future Scope

14.1 Introduction

Technology is continuously evolving, and surveillance systems are becoming more intelligent with the help of Artificial Intelligence (AI), Machine Learning (ML), the Internet of Things (IoT), and cloud computing. Although the current **Motion Detection Using CCTV Camera with Python** project successfully detects motion in real time, it can be further enhanced by adding advanced features to improve accuracy, automation, and overall performance.

14.2 Future Enhancements

The following improvements can be made in future versions of the project:

1. Artificial Intelligence-Based Object Detection

Artificial Intelligence can be integrated to identify and classify moving objects such as humans, vehicles, animals, or other objects instead of detecting only motion.

2. Face Recognition

The system can be enhanced with face recognition technology to identify authorized and unauthorized persons. This feature can be useful in offices, schools, banks, and other secure locations.

3. Automatic Video Recording

The system can automatically start recording whenever motion is detected and stop recording when no movement is present. This helps save storage space.

4. Image Capture

The application can automatically capture images of moving objects and save them with the date and time for future reference.

5. Mobile Notifications

The system can send instant notifications to a smartphone whenever motion is detected, allowing users to monitor their property remotely.

6. Email and SMS Alerts

Email or SMS alerts can be integrated so that users receive immediate notifications during suspicious activities.

7. Cloud Storage Integration

Captured images and recorded videos can be stored securely in cloud services, making them accessible from anywhere.

8. Multiple Camera Support

The project can be extended to monitor multiple CCTV cameras simultaneously from a single application.

9. IoT-Based Smart Surveillance

Integration with Internet of Things (IoT) devices can enable remote monitoring, smart home automation, and centralized surveillance management.

10. Improved Detection Accuracy

Advanced image processing and Machine Learning algorithms can reduce false detections caused by lighting changes, shadows, rain, or camera vibrations.

11. Cross-Platform Compatibility

Future versions can be developed to work on Windows, Linux, macOS, Android, and other operating systems without major modifications.

12. Web-Based Monitoring Dashboard

A web application can be developed to allow users to monitor live camera feeds, review recorded videos, and manage surveillance settings through a browser.

14.3 Benefits of Future Enhancements

The proposed improvements will provide several benefits, including:

- Higher detection accuracy.
- Reduced false alarms.
- Remote monitoring from anywhere.
- Better data security through cloud storage.
- Automatic notifications and alerts.
- Support for multiple cameras.
- Intelligent object recognition.
- Improved scalability and flexibility.
- Enhanced user convenience and system reliability.

14.4 Summary

The current motion detection system provides an effective and affordable solution for basic surveillance applications. However, by integrating modern technologies such as Artificial Intelligence, Machine Learning, IoT, cloud computing, face recognition, and mobile applications, the system can be transformed into a smart and intelligent surveillance solution. These future enhancements will improve security, increase automation, and make the system suitable for advanced real-world applications.

Chapter 15 – Conclusion

15.1 Conclusion

The **Motion Detection Using CCTV Camera with Python** project was successfully designed, developed, and implemented to detect motion in real time using computer vision techniques. The system captures live video from a webcam or CCTV camera, compares consecutive frames, and identifies moving objects by drawing a bounding box around them. This demonstrates the practical application of Python programming and the OpenCV library in developing an efficient surveillance system.

The project successfully achieved its main objectives by providing a simple, reliable, and cost-effective motion detection solution. It reduces the need for continuous manual monitoring and helps improve security by automatically identifying movement within the camera's field of view. The use of image processing techniques such as grayscale conversion, Gaussian Blur, frame differencing, thresholding, and contour detection enabled accurate motion detection under normal operating conditions.

One of the major advantages of this project is its simplicity and affordability. It can be implemented using a standard webcam or CCTV camera without requiring expensive hardware or specialized equipment. This makes it suitable for educational institutions, homes, offices, banks, hospitals, warehouses, parking areas, and other locations where basic surveillance is required.

Although the current system has some limitations, such as sensitivity to lighting conditions and the inability to identify specific objects, it provides a strong foundation for future development. By integrating advanced technologies such as Artificial Intelligence (AI), Machine Learning (ML), face recognition, cloud storage, mobile notifications, and IoT, the system can be transformed into a more intelligent and automated surveillance solution.

Overall, this project demonstrates that motion detection using Python and OpenCV is an effective approach for real-time surveillance. It provides valuable practical knowledge in Python programming, computer vision, and image processing while offering a reliable security solution. The successful implementation of this project also serves as an excellent learning experience and a foundation for developing advanced smart surveillance systems in the future.

Chapter 16 – References

16.1 Introduction

The following books, research papers, online resources, and software documentation were referred to during the development of the **Motion Detection Using CCTV Camera with Python** project. These references provided valuable information about Python programming, computer vision, image processing, motion detection techniques, and OpenCV implementation.

16.2 Books

1. Bradski, G., & Kaehler, A. *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
2. Szeliski, R. *Computer Vision: Algorithms and Applications*. Springer.
3. Zelle, J. M. *Python Programming: An Introduction to Computer Science*. Franklin, Beedle & Associates.
4. Downey, A. B. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

16.3 Research Papers

1. Research papers on Motion Detection using Computer Vision published in IEEE Xplore.
2. Research articles on Video Surveillance Systems published in Springer.
3. Journals related to Image Processing and Computer Vision.
4. Articles on Artificial Intelligence and Machine Learning for Smart Surveillance Systems.

16.4 Online Resources

1. Python Official Documentation – <https://docs.python.org/>
2. OpenCV Official Documentation – <https://docs.opencv.org/>
3. NumPy Official Documentation – <https://numpy.org/doc/>
4. Visual Studio Code Documentation – <https://code.visualstudio.com/docs>
5. Python Package Index (PyPI) – <https://pypi.org/>

16.5 Software Used

- Python 3.x
- OpenCV Library
- NumPy Library
- Visual Studio Code
- Windows 10/11
- Webcam/CCTV Camera

16.6 Summary

The above references were used to understand the concepts of Python programming, computer vision, image processing, and motion detection. They also helped in the successful implementation of the project using Python and OpenCV. These sources provide reliable information and can be referred to for future study and further improvements to the project.